RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

AD23431 - STATISTICAL ANALYSIS AND COMPUTING

EXP 1: Implementation of Vectors, Matrices, and Arrays in R

I Vector Operations in R

"The Daily Grind" Sales Analysis

Analyze the sales performance of a local café over six months. You have data for two main product categories: **Coffee** and **Pastries**.

Using Vectors in R do the following:

a. Create the datasets representing monthly revenue (in USD) from January to June.

months<-c("Jan","Feb","Mar","Apr","May","Jun")
coffee<-c(10000,2000,300,20,100,3000)
pest<-c(3000,200,100,2000,300,100)

b. Calculate the total revenue for the cafe each month (Coffee + Pastries) and the overall total for the half-year.

totalforeachmonth<-coffee+pest
totalforeachmonth

Output:

```
13000  2200   400  2020   400  3100
```

c. Find the average monthly sales for coffee and determine the standard deviation to see how much sales fluctuate.

average<-mean(coffee)
standarddeviation<-sd(coffee)
average
standarddeviation

Output:

```
> average
 2570
> standarddeviation
 3833.301
```

d. The owner wants to know which months the coffee sales exceeded **$3,000**. Use logical indexing to extract these months.

monthsexceeds3000<-months[coffee>3000]
monthsexceeds3000

Output:

```
"Jan"
```

e. Calculate the difference in sales between June and January to see the growth over the period.

differencecoffee<-coffee[6]-coffee[1]
differencepest<-pest[6]-pest[1]
differencecoffee
differencepest

Output:

```
> differencecoffee
[1] -7000
> differencepest
[1] -2900
```

II Vector Operations in R

"FitLife Gym" Membership Revenue Analysis

Analyze the monthly membership revenue of a fitness center for six months.

Using Vectors in R, do the following:

a. Create vectors representing monthly revenue (in USD) from **Basic** and **Premium** memberships (January–June).

months<-c("Jan","Feb","Mar","Apr","May","Jun")
basic<-c(2000,3000,4000,4000,5000,6000)
premium<-c(3000,4000,5000,5000,6000,7000)

b. Calculate the total revenue for each month and the overall revenue for the six-month period.

total<-basic+premium
months
total
sum(total)

Output:

> months
[1] "Jan" "Feb" "Mar" "Apr" "May" "Jun"
> total
[1] 5000 7000 9000 9000 11000 13000
> sum(total)
[1] 54000

c. Find the average monthly revenue for Premium memberships and compute the standard deviation.

sum(premium)/length(months)
sd(premium)

Output:

> sum(premium)/length(months)
[1] 5000
> sd(premium)
[1] 1414.214

d. Identify the months in which Premium membership revenue exceeded **$5,000** using logical indexing.

months[premium>5000]

Output:

[1] "May" "Jun"

e. Calculate the revenue growth from January to June for Basic memberships.

revenue_growth<-basic[6]-basic[1]
revenue_growth

Output:

> revenue_growth
[1] 4000

III Matrix Operations in R

"TechStore" Quarterly Sales Performance

A technology store records quarterly sales of **Laptops, Mobiles, and Accessories** across **three branches**.

Using Matrices in R, do the following:

a. Create a matrix representing sales data where rows correspond to branches and columns represent product categories.

```
sales<-
matrix(c(50000,30000,20000,60000,40000,25000,55000,35000,22000),
nrow=3,byrow=TRUE)
rownames(sales)<-c("Branch1","Branch2","Branch3")
colnames(sales)<-c("Laptops","Mobiles","Accessories")
sales
```

Output:

```
> sales
        Laptops Mobiles Accessories
Branch1   50000   30000       20000
Branch2   60000   40000       25000
Branch3   55000   35000       22000
```

b. Calculate the total sales for each branch and each product.

```
branch_total<-rowSums(sales)
branch_total
product_total<-colSums(sales)
product_total
```

Output:

```
> branch_total
Branch1 Branch2 Branch3
 100000  125000  112000
```

> product_total

```
   Laptops     Mobiles Accessories
   165000      105000       67000
```

c. Find the branch with the highest total sales.

names(branch_total)[which.max(branch_total)]

<u>Output:</u>

[1] "Branch2"

d. Compute the average sales for each product category.

product_average<-colMeans(sales)

product_average

<u>Output:</u>

> product_average

```
   Laptops     Mobiles Accessories
  55000.00    35000.00    22333.33
```

e. Access and display sales of **Mobiles** in the **second branch**.

sales[2,"Mobiles"]

<u>Output:</u>

> sales[2,"Mobiles"]

[1] 40000

IV Matrix Operations in R

Student Marks Analysis

A class has marks in **Mathematics, Physics, and Computer Science** for **five students**.

Using Matrices in R, do the following:

a. Create a matrix to store student marks.

marks<-matrix(c(80,75,90,70,65,85,88,82,91,60,72,68,95,90,93),nrow=5

```
,byrow=TRUE)
rownames(marks)<-c("S1","S2","S3","S4","S5")
colnames(marks)<-c("Maths","Physics","CS")
marks
```

Ouput:

```
> marks
  Maths Physics CS
S1    80      75 90
S2    70      65 85
S3    88      82 91
S4    60      72 68
S5    95      90 93
```

b. Calculate the total and average marks for each student.

```
total<-rowSums(marks)
total
average<-rowMeans(marks)
average
```

Output:

```
> total
 S1  S2  S3  S4  S5
245 220 261 200 278
> average
      S1       S2       S3       S4       S5
81.66667 73.33333 87.00000 66.66667 92.66667
```

c. Find the subject-wise highest and lowest marks.

```
apply(marks,2,max)
apply(marks,2,min)
```

Output:

```
  Maths Physics      CS
     95      90      93
  Maths Physics      CS
```

60 65 68

d. Identify students who scored more than **75** in all subjects.

students_above75<-rownames(marks)[apply(marks>75,1,all)]
students_above75

Output:

[1] "S3" "S5"

e. Replace marks below **40** with **"Fail"** indication (using logical conditions).

marks_fail<-as.character(marks)
marks_fail[as.numeric(marks_fail)<40]<-"Fail"
marks_fail<-matrix(marks_fail,nrow=nrow(marks))
rownames(marks_fail)<-rownames(marks)
colnames(marks_fail)<-colnames(marks)

marks_fail

Output:

```
> marks_fail
  Maths Physics CS
S1 "80"  "75"    "90"
S2 "70"  "65"    "85"
S3 "88"  "82"    "91"
S4 "60"  "72"    "68"
S5 "95"  "90"    "93"
```

V Array Operations in R

Hospital Patient Statistics

A hospital records the number of patients across **3 departments**, **4 weeks**, and **2 shifts (Day/Night)**.

Using Arrays in R, do the following:

a. Create a 3-dimensional array to store patient counts.

```
patients<-array(c(
  30,25,20,   40,35,30,
  28,22,18,   38,32,26,
  35,30,25,   45,40,35
),dim=c(3,3,2))

dimnames(patients)<-list(
  Department=c("OPD","ICU","Ward"),
  Shift=c("Morning","Evening","Night"),
  Week=c("Week1","Week2")
)
patients
```

Output:

```
 Week1
     Morning Evening Night
OPD       30      25    20
ICU      40      35    30
Ward      28      22    18

 Week2
     Morning Evening Night
OPD       38      32    26
ICU      35      30    25
Ward      45      40    35
```

b. Calculate the total number of patients per department.

apply(patients,1,sum)

<u>Output:</u>

OPD ICU Ward
171 195 188

c. Find the average number of patients per week.

apply(patients,3,mean)

<u>Output:</u>

Week1 Week2
26.44 34.56

d. Extract patient data for the **Night shift** only.

patients[, "Night",]

<u>Output:</u>

```
     Week1 Week2
OPD    20    26
ICU    30    25
Ward   18    35
```

e. Identify the week with the highest patient count.

names(which.max(apply(patients,3,sum)))

<u>Output:</u>

"Week2"

VI Array Operations in R

Weather Data Analysis

A meteorological department collects temperature data for **3 cities**, **7 days**, and **2 time periods (Morning/Evening)**.

Using Arrays in R, do the following:

a. Create an array to store the temperature data.

```
temp<-array(c(
  30,28,  31,29,  32,30,
  25,23,  26,24,  27,25,
  35,33,  36,34,  37,35
),dim=c(3,3,2))
dimnames(temp)<-list(
  City=c("CityA","CityB","CityC"),
  Day=c("Day1","Day2","Day3"),
  Time=c("Morning","Evening")
)
```

b. Calculate the average temperature for each city.

```
apply(temp,1,mean)
```

Output:

```
CityA CityB CityC
 30    25    35
```

c. Determine the day with the highest recorded temperature.

daymax<-apply(temp,2,max)
names(daymax)[which.max(daymax)]
<u>Output:</u>
"Day3"

d. Extract temperature data for a specific city across all days.
temp["CityA",,]
<u>Output:</u>

	Morning	Evening
Day1	30	28
Day2	31	29
Day3	32	30

e. Compare morning and evening temperatures using array slicing.
temp[,,"Morning"]-temp[,,"Evening"]
<u>Output:</u>

	Day1	Day2	Day3
CityA	2	2	2
CityB	2	2	2
CityC	2	2	2